



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №7

із дисципліни «Технології розробки програмного забезпечення»

Тема: «ШАБЛОНИ «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD»

Виконав:
Студент групи ІА-24
Зелінський І.О.

Перевірив:
Мякий М. Ю.

Зміст

Тема.....	3
Мета.....	3
Завдання.....	3
Теоретичні відомості.....	3
Обрана тема.....	5
Клас OnlineRadioFacade	6
Використання AudioStreamFactory.....	8
Висновок.....	9

Тема: Шаблони «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD»

Мета: Навчитися реалізовувати шаблони «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD»

Завдання:

1. Реалізувати не менше 3-х класів відповідно до обраної теми.
2. Реалізувати один з розглянутих шаблонів за обраною темою.

Теоретичні відомості

Шаблони проєктування

Шаблон проєктування — це типовий підхід до вирішення поширених проблем, які виникають під час створення архітектури програмного забезпечення. Вони не є готовими бібліотеками чи функціями, які можна просто скопіювати в проєкт. Замість цього шаблон описує загальний принцип, який потребує адаптації до конкретних умов програми.

Найпростіші шаблони, які мають сенс лише в певній мові програмування, називають *ідіомами*. Найбільш універсальні — це *архітектурні шаблони*, які можна застосувати в будь-якій мові програмування. Їх використовують для створення структури програми в цілому, а не окремих її частин.

Класифікація шаблонів

Шаблони можна поділити на три основні групи:

1. **Породжуючі:** допомагають створювати об'єкти, зменшуючи кількість залежностей у програмі.
2. **Структурні:** визначають, як об'єкти можуть взаємодіяти між собою.
3. **Поведінкові:** оптимізують обмін даними та комунікацію між об'єктами.

1. Шаблон «Mediator» (Посередник)

Шаблон "Посередник" спрямований на зменшення прямої взаємодії між об'єктами, замінюючи її спілкуванням через центральний об'єкт – посередника. Завдяки цьому об'єкти стають менш залежними один від одного, що полегшує їх модифікацію та розширення. Посередник містить логіку координації та управління взаємодією між

об'єктами. Типовим прикладом є система обробки подій, де різні компоненти інтерфейсу передають інформацію через посередника. Цей підхід особливо корисний у випадках, коли кількість компонентів зростає, і їхні зв'язки стають складними. Недолік – посередник може стати надто складним, якщо вся логіка переміщується до нього. Реалізація вимагає ретельного планування, щоб уникнути надмірного збільшення складності посередника.

2. Шаблон «Facade» (Фасад)

Фасад надає спрощений інтерфейс до складної підсистеми, приховуючи її складність від клієнтів. Основна мета – зменшення залежності між клієнтами і компонентами системи, полегшуючи використання. Він забезпечує єдиний вхідний пункт до системи, що спрощує її тестування, модифікацію та розуміння. Використовується, коли система має багато взаємопов'язаних класів або складну логіку. Наприклад, у програмі для роботи з графікою фасад може надати прості методи для створення і редагування зображень, приховуючи складні математичні операції. Хоча фасад спрощує доступ, важливо не допустити дублювання логіки в фасаді та підсистемах. Це популярний шаблон для створення API або бібліотек.

3. Шаблон «Bridge» (Міст)

Шаблон "Міст" розділяє абстракцію та її реалізацію, дозволяючи їм змінюватися незалежно один від одного. Це досягається введенням базового інтерфейсу (або абстракції) та відділенням від нього реалізацій через композицію. Наприклад, у програмі для малювання різні типи фігур (абстракція) можуть використовувати різні способи рендерингу (реалізація). Це сприяє зменшенню кількості підкласів, адже комбінації абстракцій і реалізацій тепер створюються динамічно. Шаблон корисний у випадках, коли класи часто змінюються або мають кілька варіантів поведінки. "Міст" покращує розширюваність системи, але може збільшити складність через додаткові класи. Його часто використовують у графічних або апаратно-незалежних додатках.

4. Шаблон «Template Method» (Шаблонний метод)

Цей шаблон визначає "скелет" алгоритму в базовому класі, дозволяючи підкласам реалізовувати конкретні кроки цього алгоритму. Це забезпечує повторне використання

загальної структури алгоритму, уникаючи дублювання коду. Наприклад, у системі обробки документів базовий клас може містити методи зчитування, аналізу та збереження файлу, тоді як підкласи реалізують специфіку роботи з PDF чи Word. Шаблон корисний, коли необхідно визначити чіткий порядок виконання операцій із можливістю їхнього часткового перевизначення. Його перевага – гнучкість і простота підтримки загальної логіки. Недолік – обмежена свобода дій для підкласів, адже вони повинні слідувати жорстко заданій структурі алгоритму. Часто використовується для створення фреймворків.

Обрана тема: Online radio station (iterator, adapter, factory method, facade, visitor, client-server)

Додаток повинен служити сервером для радіостанції з можливістю мовлення на радіостанцію (64, 92, 128, 196, 224 kb/s) в потоковому режимі; вести облік підключених користувачів і статистику відвідувань і прослуховувань; налаштувати папки з піснями і можливість вести списки програвання або playlists (не відтворювати всі пісні).

Хід роботи

Під час розробки програми я реалізував шаблон "Facade", оскільки проєкт “Online radio station” має бути добре структурований і Facade забезпечує зручність використання, підтримки й масштабування системи онлайн-радіо.

Переваги фасаду OnlineRadioFacade:

- Інкапсуляція логіки бізнес-процесів

Фасад абстрагує взаємодію з різними сервісами (UserService, PlaylistService, SongService, StreamingService, ArtistService, StatisticService). Це полегшує роботу з ними в контролерах чи інших частинах системи, знижуючи залежність від деталей реалізації сервісів.

- Зниження зв'язності

Код клієнтів (наприклад, контролери) залежить тільки від фасаду, а не від безлічі окремих сервісів. Це спрощує підтримку і розвиток, оскільки зміни в сервісах не впливають безпосередньо на клієнтів.

- Єдиний інтерфейс

OnlineRadioFacade забезпечує єдину точку доступу до функціоналу радіостанції. Це робить код більш структурованим і легшим для розуміння.

- Проста розширюваність

Додати нові методи або змінити існуючу логіку можна, змінюючи лише фасад і пов'язані сервіси, без зміни клієнтського коду.

Клас **OnlineRadioFacade**:

```
package com.OnlineRadio.OnlineRadioStation.facade;

import com.OnlineRadio.OnlineRadioStation.models.*;
import com.OnlineRadio.OnlineRadioStation.services.*;
import org.springframework.stereotype.Service;
import org.springframework.beans.factory.annotation.Autowired;
import java.util.ArrayList;
import java.util.List;

@Service
public class OnlineRadioFacade {
    private final UserService userService;
    private final PlaylistService playlistService;
    private final SongService songService;
    private final StreamingService streamingService;
    private final ArtistService artistService;
    private final StatisticService statisticService;

    public OnlineRadioFacade(UserService userService, PlaylistService
playlistService,
                           SongService songService, StreamingService
streamingService,
                           ArtistService artistService, StatisticService
statisticService) {
        this.userService = userService;
        this.playlistService = playlistService;
        this.songService = songService;
        this.streamingService = streamingService;
        this.artistService = artistService;
        this.statisticService = statisticService;
    }

    public User login(String login, String password) {
        return userService.login(login, password);
    }

    public boolean register(String login, String password, String
ipAddress, String roleName) {
        return userService.register(login, password, ipAddress, roleName);
    }

    public Playlist createPlaylist(String name, String ownerId,
Playlist.Status status) {
        return playlistService.createPlaylist(name, ownerId, status);
    }

    public void deletePlaylist(String playlistId) {
        playlistService.deletePlaylist(playlistId);
    }

    public void addSongToPlaylist(String playlistId, String songId) {
```

```

        playlistService.addSongToPlaylist(playlistId, songId);
    }

    public void removeSongFromPlaylist(String playlistId, String songId) {
        playlistService.removeSongFromPlaylist(playlistId, songId);
    }

    public List<Song> getSongsFromPlaylist(String playlistId) {
        List<String> songIds =
playlistService.getSongIdsInPlaylist(playlistId);
        List<Song> songs = new ArrayList<>();
        for (String songId : songIds) {
            Song song = songService.findSongById(songId);
            if (song != null) {
                songs.add(song);
            }
        }
        return songs;
    }

    public void stopStreaming() {
        streamingService.stopStreaming();
    }

    public void playSong(String songId, String quality) {
        Song song = songService.findSongById(songId);
        if (song != null) {
            String songPath = song.getFilePath();
            streamingService.startStreaming(songPath, quality);
        } else {
            System.out.println("Song not found with id: " + songId);
        }
    }

    public List<Song> getAllSongs() {
        return songService.getAllSongs();
    }

    public void addSong(Song song) {
        songService.addSong(song);
    }

    public void removeSong(String songId) {
        songService.removeSong(songId);
    }

    public Artist addArtist(String name) {
        return artistService.addArtist(name);
    }

    public List<Statistic> getStatistics(String userId) {
        return statisticService.getUserStatistics(userId);
    }
}

```

Рисунок 1. Клас OnlineRadioFacade

Використання OnlineRadioFacade у контролерах (приклад)

```
package com.OnlineRadio.OnlineRadioStation.controllers;

import com.OnlineRadio.OnlineRadioStation.facade.OnlineRadioFacade;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/streaming")
public class StreamingController {

    @Autowired
    private OnlineRadioFacade onlineRadioFacade;

    @PostMapping("/play/{id}")
    public String playSong(@PathVariable("id") String songId,
        @RequestParam("quality") String quality) {
        try {
            onlineRadioFacade.playSong(songId, quality);
            return "Streaming started with " + quality + " quality.";
        } catch (Exception e) {
            return "Error starting stream: " + e.getMessage();
        }
    }

    @PostMapping("/stop")
    public String stopStreaming() {
        try {
            onlineRadioFacade.stopStreaming();
            return "Streaming stopped.";
        } catch (Exception e) {
            return "Error stopping stream: " + e.getMessage();
        }
    }
}
```

Рисунок 2. Клас StreamingController

```
package com.OnlineRadio.OnlineRadioStation.controllers;

import com.OnlineRadio.OnlineRadioStation.models.Song;
import com.OnlineRadio.OnlineRadioStation.facade.OnlineRadioFacade;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/songs")
public class SongController {

    private final OnlineRadioFacade facade;

    @Autowired
    public SongController(OnlineRadioFacade facade) {
        this.facade = facade;
    }
}
```

```

@GetMapping
public List<Song> getAllSongs() {
    return facade.getAllSongs();
}

@PostMapping
public void addSong(@RequestBody Song song) {
    facade.addSong(song);
}

@DeleteMapping("/{songId}")
public void removeSong(@PathVariable String songId) {
    facade.removeSong(songId);
}
}

```

Рисунок 3. Клас SongController

Висновок: На цій лаб. роботі я зрозумів суть шаблонів «MEDIATOR», «FACADE», «BRIDGE», «TEMPLATE METHOD», реалізував один з них (FACADE) для свого проекту “Online radio station”, адже він має багато переваг для структури мого проекту, такі як інкапсуляція логіки бізнес-процесів, зниження зв’язності, логічна сегментація функціоналу й реалізація єдиного інтерфейсу.